Cuneiform Operator Language Master Specification v3.0

Deterministic Glyph Programming as a Registry-Bound, Evidence-Carrying, Evolvable Frontend for Babylon Compiler

Sebastian Klemm

March 2026

This specification absorbs and supersedes:

v1.0	*Glyph Operator Design (Babylon Compiler) — German outline*
v2.0	*Cuneiform Operator Language Specification — English*
v2.1	*Keilschrift++ / COL v2.1 — German, extended*

All three predecessor documents are hereby rendered historically obsolete.

Abstract

This document is the sole normative specification for the Cuneiform Operator Language (COL), a production-oriented glyph language designed for deterministic compilation, evidence-bearing execution, controlled language evolution, and replay verification within the Babylon Compiler. COL treats cuneiform not as an exotic surface syntax but as a *registry-bound operator address space* in which glyphs and glyph clusters address versioned operators, deterministic macro expansions, and evidence-bearing semantic transitions that lower into a single canonical intermediate representation.

The specification defines COL as a state-first, graph-first, and evidence-first system. It normatively establishes: a strict Unicode policy for cuneiform sources; a single canonical lowering target aligned with Babylon's `glyph-ir`; a family of five digest-bound registries (Sign, Macro, Profile, Obligation, Gate); a formal epoch model for frozen language versions; a deterministic mutation engine with seed-bound candidate generation; a tripolar decision layer (`0 / LD / 1`) for mutation acceptance; a contraction-based stabilization principle with Banach-fixpoint interpretation; a crystallization layer for freezing new language epochs; a resonance analysis layer for coherence evaluation; evidence capsules as declared boundary objects; and a bundle/manifest discipline that preserves replayability across evolution.

The result is not merely a new syntax but a language architecture in which the evolution of the language itself becomes a formal, inspectable, and reproducible process. Integration into Babylon Compiler is specified at the level of crate structure, CLI commands, pipeline stages, and bundle format.

Contents

1 Preamble **6**
- 1.1 Scope . . . 6
- 1.2 Normative Language . . . 6
- 1.3 Evolutionary Context . . . 6
- 1.4 Extracted Invariants . . . 7

2 Conflict, Redundancy, and Maturity Analysis **8**
- 2.1 Redundancies Resolved . . . 8
- 2.2 Conflicts Resolved . . . 8
- 2.3 Maturity Assessment . . . 9

3 Babylon Compiler Alignment **9**
- 3.1 Pipeline Mapping . . . 9
- 3.2 Crate Architecture . . . 10
- 3.3 Extension Points and Invariant Preservation . . . 11

4 Synthesis Decisions **11**

5 System Context and Theoretical Basis **13**
- 5.1 State-First Semantics . . . 13
- 5.2 Babylon Compiler as Deterministic Foundry . . . 13
- 5.3 Graph-First Program Objects . . . 13
- 5.4 Three Time Scales . . . 13

6 Unicode and Glyph Model **13**
- 6.1 Accepted Ranges . . . 13
- 6.2 Normalization Policy . . . 14
- 6.3 Glyphs as Atomic Semantic Tokens . . . 14
- 6.4 Transliteration Mode . . . 14

7 Cuneiform as an Operator Address Space **14**
- 7.1 From Surface to Addressing . . . 14
- 7.2 Non-Triviality Criteria . . . 14
- 7.3 Semantic Density Metric . . . 15

8 Core Grammar and Minimal Language Model **15**
- 8.1 Minimal Language Model . . . 15
- 8.2 Normative EBNF . . . 15

9 Registries **16**
- 9.1 Registry Family . . . 16
- 9.2 Sign Registry Detail . . . 16
- 9.3 Digest Binding . . . 16
- 9.4 Registry Evolution . . . 17

10 Language Epochs **17**
- 10.1 Definition . . . 17
- 10.2 Epoch Object . . . 17
- 10.3 Replay Stability Across Epochs . . . 17

11 Mutation Engine **17**
11.1 Candidate Generation . . . 18
11.2 Evaluation Metrics . . . 18
11.3 Mutation Formalization . . . 18
11.4 Mutation Manifest . . . 18

12 Tripolar Gate Logic **18**
12.1 Three Decision States . . . 18
12.2 Gate Semantics . . . 19
12.3 Gate Registry Integration . . . 19
12.4 Gate Object . . . 19

13 Contraction, Fixpoints, and Condensation **19**
13.1 Contractive Update Principle . . . 19
13.2 Banach Fixpoint Interpretation . . . 20
13.3 Micro-Dynamics Convergence . . . 20
13.4 Condensation . . . 20

14 HDAG Ordering and Invariance **20**
14.1 Ordered Language Lineage . . . 20
14.2 Cross-Epoch Invariants . . . 20
14.3 Self-Compilation Stability . . . 20

15 Crystallization Layer **21**
15.1 Mutation as Observation Stream . . . 21
15.2 Crystal Formation Criteria . . . 21
15.3 Crystal Manifest . . . 21
15.4 MEF and Evidence Chain . . . 21

16 Resonance Layer **21**
16.1 Operators as Resonance Network . . . 21
16.2 Resonance Gating . . . 21
16.3 Spectral Compression . . . 22

17 Spiral Embedding Model (Optional) **22**
17.1 Spiral Coordinates . . . 22
17.2 Convergence Conditions . . . 22

18 QLOGIC-ISA Layer (Optional Extension) **22**
18.1 Spectral Opcodes . . . 22
18.2 Multi-Stage Addressing . . . 22
18.3 Path Invariance . . . 23

19 Evidence Capsules **23**
19.1 Definition . . . 23
19.2 Capsule Policy . . . 23
19.3 Capsule Object . . . 23
19.4 Verify and Replay . . . 23

20 Canonical IR and Lowering Contract **24**
20.1 Single Canonical Target . . . 24

20.2 Extended Metadata . . . 24
20.3 Stable Lowering . . . 24
20.4 Deterministic ID Generation . . . 24
20.5 Backward Compatibility . . . 24

21 Integration into Babylon Compiler 25
21.1 Frontend Architecture . . . 25
21.2 CLI Extensions . . . 25
21.3 Bundle Format v3.0 . . . 25

22 Test Architecture 26
22.1 T1: Determinism Tests . . . 26
22.2 T2: Evolution Tests . . . 26
22.3 T3: Tripolar Gate Tests . . . 26
22.4 T4: Contraction and Fixpoint Tests . . . 26
22.5 T5: Crystal Tests . . . 26
22.6 T6: Capsule Tests . . . 26
22.7 Normative Test Vectors . . . 26

23 Definition of Done 27

24 Obsolescence of Predecessor Documents 28

A Epoch Object Example 28

B Mutation Manifest Example 29

C Gate Object Example 29

D Crystal Manifest Example 29

E Capsule Object Example 30

F Sign Registry v1 Example 30

G Normative Test Vector TV1: Minimal Print 31

H Run Descriptor Extension for COL 32

I Glossary 32

1 Preamble

1.1 Scope

This specification covers the complete Cuneiform Operator Language (COL) as a frontend language system for Babylon Compiler. It is normative for:

(a) the lexical and syntactic structure of COL source programs,

(b) the lowering contract from COL surface to Babylon's canonical IR,

(c) the registry family that governs glyph semantics and macro expansion,

(d) the epoch model for frozen, replayable language versions,

(e) the mutation engine for controlled language evolution,

(f) the tripolar gate logic for mutation acceptance,

(g) the contraction/fixpoint/crystallization discipline,

(h) the resonance analysis layer,

(i) evidence capsules and their policy framework,

(j) integration into Babylon Compiler's crate architecture, CLI surface, and bundle format,

(k) the test architecture and the normative Definition of Done.

1.2 Normative Language

Throughout this document, "MUST" indicates an absolute requirement; "MUST NOT" an absolute prohibition; "SHOULD" a strong recommendation; "MAY" an optional feature. Requirements are numbered as **[REQ-S.N]** where `S` is the section and `N` the ordinal.

1.3 Evolutionary Context

This specification was derived by invariant extraction, conflict resolution, and gap closure from three predecessor documents:

v1.0 (COL_de)
: German-language outline (11 pages). Established the core thesis of cuneiform as an operator address space, defined determinism, canonicality, evidence binding, the Sign Registry, the minimal EBNF grammar, the IR lowering contract, and the Definition of Done. Remained static (no evolution model) and contained mostly placeholders in its appendices.

v2.0 (English)
: Full English specification (16 pages). Extended v1.0 with: language epochs, five registry types, a mutation engine, tripolar gate logic, contraction/fixpoint analysis, crystallization, resonance layer, and evidence capsules. Provided concrete epoch objects and gate objects. Still lacked the spiral embedding model and the QLOGIC-ISA layer.

v2.1 (Keilschrift++)
: Extended German specification (29 pages). Added the spiral mutation model,

the QLOGIC-ISA opcode layer, detailed Banach-fixpoint interpretation, micro-dynamics convergence, HDAG ordering, self-compilation stability, MEF/evidence chain details for crystallization, RSS resonance with phase/twist structure, and a substantially tightened Definition of Done.

The present v3.0 absorbs all invariants from all three documents, resolves their conflicts, closes their gaps, and binds every normative statement to the real architecture of Babylon Compiler as it exists in the repository.

1.4 Extracted Invariants

The following invariants were extracted as stable across all three predecessor documents and are *non-negotiable* in this specification:

Invariant 1.1 (Determinism)**.** For any given normalized source s, run descriptor RD, epoch E_t, and registry set $\mathcal{R}$, the system MUST produce identical IR bytes, manifest digests, trace digests, and verification outcomes. Any feature that undermines this property is disallowed.

Invariant 1.2 (Canonicality)**.** Every semantically meaningful structure MUST admit a single, stable, hashable serialized form. This applies to Unicode handling, registry binding, IR serialization, node/edge ordering, and bundle manifests. The canonical serialization format is RFC 8785 (JCS) canonical JSON; the hash algorithm is SHA-256.

Invariant 1.3 (Evidence Binding)**.** No semantically relevant transition SHOULD occur as an unrecorded side effect. Relevant decisions, mutations, gates, and crystallization steps MUST produce evidence objects that are bound into the emitted bundle via the MEF chain.

Invariant 1.4 (Registry-Digest Binding)**.** All active registries MUST be bound by cryptographic digest in the run descriptor and surfaced in emitted manifests. A build is tied not merely to an input file but to a complete semantic environment.

Invariant 1.5 (Replay Stability)**.** An artifact emitted under epoch E_t MUST remain verifiable under the same digests indefinitely, even if the language later evolves to E_{t+1} or beyond. Replay stability does not require every new epoch to interpret every old source identically; it requires that earlier epochs remain reconstructible and verifiable as earlier epochs.

Invariant 1.6 (Evolution Under Constraint)**.** The language MAY evolve only through explicitly bounded mutation procedures. A build MUST remain stable; evolution MAY occur only between builds, never inside the semantic interpretation of a single build. Three time scales are strictly separated: build time, evolution time, crystallization time.

Invariant 1.7 (Single Canonical IR Target)**.** Every valid COL source MUST lower into a single canonical IR (`glyph_ir::IrDocument`). No alternative IR truth is allowed. This is the semantic anchor that makes replay, bundle verification, and epoch coexistence possible.

Invariant 1.8 (Babylon Compiler Integration)**.** COL MUST NOT destabilize Babylon's existing canonicalization, bundle generation, or verification model. COL integrates as an additional frontend within the existing 8-stage pipeline (S0–S7). Evolution-specific modules (mutation, crystallization, epoch management) extend the pipeline without breaking it.

Invariant 1.9 (Testability)**.** Every normative claim MUST be testable. Determinism tests, evolution tests, tripolar gate tests, contraction/fixpoint tests, crystal tests, and capsule tests form the minimum required test families. The golden path `run` → `bundle` → `verify` serves as the minimal proof of function.

Invariant 1.10 (Non-Triviality Proof)**.** COL MUST demonstrate that it is more than a skin frontend. This is satisfied if and only if at least one glyph or glyph cluster deterministically expands into a non-trivial, evidence-bearing operator subgraph whose expansion is visible in the bundle, manifest, and verify process. The semantic density metric $D = \frac{\#\text{IR nodes}}{\#\text{surface tokens}}$ MUST measurably exceed 1.0 for at least one normative test vector.

2 Conflict, Redundancy, and Maturity Analysis

2.1 Redundancies Resolved

Topic	Redundancy	Resolution in v3.0
Design principles	Determinism, canonicality, evidence binding restated in all three documents with near-identical wording.	Single normative statement as Invariants 1.1–1.3.
State-first semantics	Repeated in introduction of all three documents.	Single statement in section 5.
Operator address space thesis	Central claim appears in every document.	Stated once in section 7 with formal criteria.
EBNF grammar	v1.0 mentions it; v2.0 provides full grammar; v2.1 does not repeat.	Single normative EBNF in section 8.
Discussion/conclusion	Every document contains a discursive section.	Absorbed into rationale blocks; no separate discussion section.

2.2 Conflicts Resolved

Topic	Conflict	Resolution
Registry taxonomy	v1.0 defines only a Sign Registry. v2.0/v2.1 define five registries (Sign, Macro, Profile, Obligation, Gate).	Five registries are normative. Aligned with Babylon's existing `glyph-registry` crate (Operator, Macro, Obligation, Observable) by mapping: Sign → Operator (extended), Profile → Observable (extended), Gate → new.
Epoch tuple	v2.0: $E = (R_s, R_m, R_p, R_o, R_g, P, L, C)$. v2.1: $E = (R_s, R_m, R_p, R_o, R_g, P, L)$ with C separate.	v3.0 adopts v2.1's separation: epoch and crystal are distinct objects. Crystal references its parent epoch.
Minimal role set	v1.0: FN, IF, ELSE, RETURN, LET, PRINT, TRUE, FALSE. v2.0/v2.1: implicit via grammar.	Explicit minimal role set in Sign Registry (section 9.2) matching Babylon's existing keyword sets.
Evolution model	v1.0 is static. v2.0 introduces evolution. v2.1 adds spiral model.	Evolution is normative. Spiral embedding is formalized as optional analytical layer (section 17), not as a hard pipeline requirement.

Topic	Conflict	Resolution
QLOGIC-ISA	Only in v2.1, optional.	Formalized as optional extension point (section 18). Not required for Definition of Done v3.0.
Resonance layer	v2.0 introduces; v2.1 elaborates with RSS/twist/phase.	Formalized as analytical metric contributor for gate decisions (section 16). Resonance fitness is one of several evaluation metrics; it does not replace deterministic invariants.

2.3 Maturity Assessment

Feature	v1.0	v2.0/v2.1	v3.0 Status
Core frontend (lex/-parse/lower)	Outlined	Specified	Normative, implementation-ready
Unicode policy	Outlined	Specified	Normative, aligned to `glyph-rd`
Sign/Operator Registry	Basic	Full	Normative, mapped to `glyph-registry`
Macro expansion	Mentioned	Specified	Normative, mapped to `glyph-expand`
Epoch model	Absent	Full	Normative
Mutation engine	Absent	Full	Normative
Tripolar gate logic	Absent	Full	Normative, mapped to `glyph-gate`
Contraction/fixpoint	Absent	Full	Normative
Crystallization	Absent	Full	Normative, mapped to `glyph-crystal`/`glyph-tic`
Resonance layer	Absent	Introduced	Normative as metric, not as pipeline stage
Spiral embedding	Absent	v2.1 only	Optional analytical layer
QLOGIC-ISA	Absent	v2.1 only	Optional extension point
Evidence capsules	Mentioned	Full	Normative
Bundle/verify	Mentioned	Specified	Normative, aligned to `glyph-bundle`/`glyph-verify`
Test architecture	Basic	Full	Normative, six test families
Definition of Done	Basic	Full	Normative, twelve acceptance criteria

3 Babylon Compiler Alignment

3.1 Pipeline Mapping

Babylon Compiler implements an 8-stage deterministic pipeline. COL maps onto this pipeline as follows:

Stage	Name	COL Involvement
S0	Ingest	Unicode normalization per active epoch's Unicode policy. Cuneiform ranges U+12000–U+123FF and U+12400–U+1247F accepted; all other non-whitespace forbidden in strict mode.
S1	Parse & Canonicalize	COL lexer tokenizes cuneiform signs as `SIGN(cp)`. COL parser builds AST per EBNF (section 8). COL lowering module produces `glyph_ir::IrDocument`.
S2	Expand	COL Macro Registry entries processed by `glyph-expand`. Glyph clusters expand to IR subgraphs.
S3	Embed	Standard H5 embedding via `glyph-embed`. COL-specific metadata (epoch_id, registry digests) attached.
S4	Crystallize	Standard crystallization via `glyph-crystal`/`glyph-tic`. For epoch crystallization (evolution time), the separate crystallization layer (section 15) applies.
S5	Gate	Obligation evaluation via `glyph-gate`. Tripolar gate logic (section 12) extends the existing binary pass/fail to 0/LD/1 for evolution decisions.
S6	MEF	Standard MEF chain recording. Evolution-specific evidence objects appended for mutation and crystallization steps.
S7	Emit	Bundle emission via `glyph-bundle`. Extended with epoch manifest, mutation manifest, registry lineage, crystal manifest, and capsule records.

3.2 Crate Architecture

COL integration requires the following additions and extensions to Babylon's crate workspace:

Crate / Module	Description
`glyph-frontends::cuneiform`	New frontend module containing: `lexer` (cuneiform-aware tokenizer), `parser` (EBNF implementation), `lower` (AST → IR lowering), `registry_adapter` (sign/macro resolution against digest-bound registries).
`glyph-registry` (extended)	Extended with `SignRegistry` (maps cuneiform signs → roles/operators), `ProfileRegistry` (metric profiles), and `GateRegistry` (tripolar decision rules). These supplement the existing `OperatorRegistry`, `MacroRegistry`, `ObligationRegistry`, and `ObservableRegistry`.

`glyph-epoch` (new)	Epoch management: epoch object construction, epoch lineage (HDAG), epoch serialization, epoch digest computation.
`glyph-mutate` (new)	Mutation engine: deterministic candidate generation, metric evaluation, mutation manifest emission.
`glyph-gate` (extended)	Extended with tripolar state assignment (0/LD/1), gate registry evaluation, evidence template instantiation.
`glyph-crystal` (extended)	Extended with epoch crystallization: crystal formation criteria, crystal manifest emission, TIC-based convergence for language-level stabilization.
`glyph-capsule` (new)	Evidence capsule management: policy enforcement, size/type constraints, hash binding, provenance tracking.
`glyph-bundle` (extended)	Extended manifest schema with `epoch_id`, five registry digests, `mutation_lineage`, `crystal_id`, `capsule_ids`.
`glyph-cli` (extended)	New subcommands: `evolve-registry`, `crystallize-epoch`, `inspect-epoch`. Existing `run` gains `--frontend cuneiform` and `--epoch` flags.

3.3 Extension Points and Invariant Preservation

Requirement 3.1 (Frontend Trait Compliance)**.** **[REQ-3.1]** The cuneiform frontend MUST implement the same interface pattern as `sanskroot` and `hanlan`: a `lex()` function returning a token stream, a `parse()` function returning an AST, and a `lower()` function producing an `IrDocument`. The CLI dispatches by matching `frontend == "cuneiform"` in `s1_canon()`.

Requirement 3.2 (No Pipeline Breakage)**.** **[REQ-3.2]** All existing pipeline stages (S0–S7) MUST continue to function identically for `sanskroot` and `hanlan` frontends after COL integration. COL-specific code paths MUST NOT alter shared pipeline logic.

Requirement 3.3 (Registry Digest Compatibility)**.** **[REQ-3.3]** New registry types (Sign, Profile, Gate) MUST use the same `compute_digest()` function from `glyph-registry` (canonical JSON → SHA-256). Their digests MUST be included in the `RunDescriptor.registries` struct and surfaced in the bundle manifest.

4 Synthesis Decisions

The following decisions were taken to resolve ambiguities and close gaps left open by the predecessor documents:

SD-1 Registry naming alignment. The v2.0/v2.1 "Sign Registry" is semantically closest to Babylon's `OperatorRegistry` but requires cuneiform-specific fields (Unicode codepoint, glyph cluster pattern, transliteration alias). v3.0 defines `SignRegistry` as a new struct that *extends* the operator concept

with glyph-specific addressing. At serialization time, it produces a superset of `OperatorEntry` fields.

SD-2 Profile vs. Observable. The v2.0/v2.1 "Profile Registry" (metric profiles, evaluation functions) maps to Babylon's `ObservableRegistry` (arity, deterministic flag, computation reference). v3.0 unifies these: profile entries are registered as observables with additional metric-weighting metadata.

SD-3 Gate Registry as new type. No direct Babylon equivalent exists. v3.0 defines `GateRegistry` as a new registry type within `glyph-registry`, containing gate definitions with inputs, thresholds, tripolar mapping rules, and evidence templates.

SD-4 Epoch as first-class object. Babylon's current pipeline does not have epoch management. v3.0 introduces `glyph-epoch` as a new crate. An epoch is a hashable, versionable, replayable JSON object. The run descriptor gains an `epoch_id` field.

SD-5 Mutation engine as separate crate. Evolution logic does not belong inside the frontend or the pipeline. v3.0 places it in `glyph-mutate`, invoked by dedicated CLI commands, never by `glyph run`.

SD-6 Spiral embedding: formalized but optional. v2.1's spiral model (ψ, ρ, ω coordinates) is formalized as an analytical frame for mutation evaluation. It is not a required pipeline stage. Implementations MAY use flat metric vectors instead, provided the evaluation metrics (section 11.2) are computed.

SD-7 QLOGIC-ISA: extension point, not v3.0 requirement. The spectral opcode layer is defined as an extension interface (section 18). It is not required for the v3.0 Definition of Done.

SD-8 Resonance: metric contributor, not separate pipeline stage. Resonance fitness contributes to gate evaluation metrics alongside coherence, entropy, drift, path-invariance gap, and compression. It does not constitute a separate pipeline stage.

SD-9 Capsule policy: explicit, bounded, not hidden. Evidence capsules are formal boundary objects with strict policy (size, type, provenance, hash). They are never hidden channels.

SD-10 Backward compatibility: epoch-scoped. Old artifacts remain verifiable under their original epoch. New epochs MUST NOT overwrite or destroy old epochs. Forward interpretation of old sources under new epochs is *not* guaranteed.

SD-11 Transliteration: auxiliary, not normative. A transliteration mode MAY exist for authoring convenience. It MUST lower deterministically to the same normalized glyph stream as native glyph input. It is never a second language.

SD-12 Self-compilation stability: strong criterion for mature epochs. A mature epoch SHOULD be able to verify its own lineage. This is a quality criterion, not a v3.0 hard gate.

5 System Context and Theoretical Basis

5.1 State-First Semantics

COL is founded on a state-first view of computation. A program is not primarily a text but an operator composition acting on a typed state space. Text is one possible input encoding; semantics reside in the structured transformation of state, not in the literal surface form. This enables multiple surfaces (Devanagari/Sanskroot, Chinese/HanLan, Cuneiform/COL) to coexist while preserving a single semantic core.

5.2 Babylon Compiler as Deterministic Foundry

Babylon Compiler provides the deterministic pipeline for ingestion, canonicalization, expansion, embedding, crystallization, gating, evidence generation, and bundle emission. COL uses this architecture as a *foundry*: the pipeline produces canonical artifacts, manifests, and verification targets rather than opaque build outputs. All identity-bearing computation uses Q16 fixed-point arithmetic (signed 32-bit, 1/65536 resolution) and RFC 8785 canonical JSON.

5.3 Graph-First Program Objects

A program is treated as a graph object (typed nodes and edges in `glyph_ir::IrDocument`) rather than a linear instruction list. This graph-first view extends naturally to language evolution: language versions, registry states, and mutation histories are all graph-structured objects subject to ordering, evidence, and replay.

5.4 Three Time Scales

Requirement 5.1 (Temporal Separation)**.** **[REQ-5.1]** COL MUST strictly separate three time scales:

(i) **Build time**: ordinary compilation under a fixed epoch and fixed registries. Deterministic and replay-stable.

(ii) **Evolution time**: generation and evaluation of candidate mutations. Occurs between builds, never during a build.

(iii) **Crystallization time**: stabilization of an accepted candidate into a new epoch. Produces a crystal manifest and a new epoch object.

Without this separation, either builds become non-deterministic or evolution collapses into arbitrary manual editing.

6 Unicode and Glyph Model

6.1 Accepted Ranges

Requirement 6.1 (Cuneiform Unicode Ranges)**.** **[REQ-6.1]** COL defines a strict Unicode policy over cuneiform source text. The accepted ranges are:

- Cuneiform: U+12000–U+123FF
- Cuneiform Numbers and Punctuation: U+12400–U+1247F
- Early Dynastic Cuneiform: U+12480–U+1254F (optional, epoch-gated)
- ASCII structural delimiters: `( ) { } , ; = > < + - * / ! "`

- ASCII digits: 0–9 (for numeric literals)
- Whitespace: U+0020, U+000A, U+000D, U+0009

In strict mode, all other non-whitespace code points are forbidden unless the active run descriptor explicitly permits them. The `glyph-rd::UnicodePolicy.forbidden_ranges` mechanism enforces this at S0.

6.2 Normalization Policy

Requirement 6.2 (Unicode Normalization)**.** **[REQ-6.2]** The source text MUST be normalized according to the active Unicode policy in the run descriptor. The default policy is NFC. The normalized source MUST be emitted into the bundle as `source.norm.txt`. This prevents multiple canonically equivalent byte sequences from producing divergent digests.

6.3 Glyphs as Atomic Semantic Tokens

Requirement 6.3 (Atomic Glyph Tokens)**.** **[REQ-6.3]** A cuneiform sign is treated as an atomic glyph token, not as an alphabetic letter. In the lexer, the normative token class is `SIGN(cp)` where `cp` is the Unicode scalar value. This prevents the language from inheriting alphabetic parsing assumptions and enables glyph-level semantic addressing.

6.4 Transliteration Mode

Requirement 6.4 (Transliteration)**.** **[REQ-6.4]** A transliteration mode MAY exist for authoring convenience and testing, but it is not normative. If provided, it MUST lower deterministically into the same normalized glyph stream that native glyph input would have produced. Transliteration MUST be treated as an auxiliary input layer, not as a second language.

7 Cuneiform as an Operator Address Space

7.1 From Surface to Addressing

In a conventional language, a token is interpreted as a keyword, identifier, literal, or operator symbol. In COL, a glyph or glyph cluster is interpreted as an *address into a registry-defined semantic space*. A sign may point to a role, an operator, or a macro expansion. This is the decisive shift from "exotic syntax" to "glyph-addressed semantics."

7.2 Non-Triviality Criteria

Requirement 7.1 (Operator Address Space Criteria)**.** **[REQ-7.1]** COL is more than a skin if and only if at least one of the following holds:

(i) A glyph addresses a versioned operator rather than merely replacing a keyword string.
(ii) A glyph deterministically expands into a non-trivial IR subgraph (semantic density $D > 1.0$).
(iii) A glyph participates in evidence-bearing or gate-bearing semantic transitions.
(iv) Glyph meaning is governed by digest-bound registries whose evolution remains replayable.

If none of these hold, the language reduces to stylistic substitution.

7.3 Semantic Density Metric

Definition 7.1 (Semantic Density)**.** The semantic density D of a COL program or program fragment is defined as:

$$D = \frac{\#\text{IR nodes after expansion}}{\#\text{surface tokens before expansion}}$$

A skin frontend has $D \approx 1$. COL aims for $D > 1$ through deterministic macro expansion and operator addressing while preserving replayability.

8 Core Grammar and Minimal Language Model

8.1 Minimal Language Model

The minimal executable subset of COL is intentionally aligned with Babylon's compact, stable IR. The language MUST support:

- modules, functions, blocks, conditionals (if/else), return statements, assignments (let),
- function calls, string/integer/boolean literals, binary arithmetic and comparison operators,
- the `print` built-in.

This matches the construct set of the existing Sanskroot and HanLan frontends, ensuring that all three frontends lower to the same `IrDocument` structure.

8.2 Normative EBNF

Requirement 8.1 (EBNF Grammar)**.** **[REQ-8.1]** The normative MVP grammar is:

Listing 1: COL EBNF Grammar v3.0

```
program     ::= { function }
function    ::= FN name '(' [ paramlist ] ')' '{' { statement } '}'
paramlist   ::= name { ',' name }
statement   ::= ifstmt | return | assignment | exprstmt
ifstmt      ::= IF '(' expression ')' '{' { statement } '}'
                [ ELSE '{' { statement } '}' ]
return      ::= RETURN expression ';'
assignment  ::= LET name '=' expression ';'
exprstmt    ::= expression ';'
expression  ::= comparison
comparison  ::= addition [ ( '>' | '<' | '>=' | '<=' | '==' | '!=' ) addition ]
addition    ::= term { ( '+' | '-' ) term }
term        ::= factor { ( '*' | '/' ) factor }
factor      ::= literal | name | call | '(' expression ')'
call        ::= PRINT '(' [ arglist ] ')' | name '(' [ arglist ] ')'
arglist     ::= expression { ',' expression }
literal     ::= INT | STRING | BOOL
name        ::= SIGN | SIGN name
```

The keywords `FN`, `IF`, `ELSE`, `RETURN`, `LET`, `PRINT`, `TRUE`, `FALSE` are resolved by the Sign Registry to specific cuneiform signs or sign clusters. Structural delimiters (`( ) { } , ; = > < + - * /`) remain as ASCII. This grammar is intentionally conservative; future epochs may extend it through registry evolution.

9 Registries

9.1 Registry Family

Requirement 9.1 (Five Registry Types). **[REQ-9.1]** COL requires the following digest-bound registries:

R1 Sign Registry (`SignRegistry`): Maps cuneiform signs or sign sequences to reserved roles (FN, IF, ELSE, RETURN, LET, PRINT, TRUE, FALSE) and operator identities. Each entry contains: sign codepoint(s), role or operator ID, version, optional transliteration alias.

R2 Macro Registry (`MacroRegistry`): Maps glyph patterns to deterministic expansion templates. Each entry contains: pattern, precedence, expansion graph (nodes + edges as JSON), bindings. Uses the existing `glyph-expand` engine with precedence resolution (lowest value wins, ties broken lexicographically).

R3 Profile Registry: Defines metric profiles and evaluation functions for mutation assessment. Entries specify metric identifiers, computation references, weighting vectors, and axis mappings. Implemented as an extension of `ObservableRegistry`.

R4 Obligation Registry (`ObligationRegistry`): Defines required proof objects and admissibility conditions. Uses the existing Babylon `ObligationEntry` structure with class (hard/soft), predicate, threshold, and axis index.

R5 Gate Registry (`GateRegistry`): Defines decision rules for the tripolar gate layer. Each entry contains: gate ID, version, input metrics, threshold functions, tripolar mapping rule (metric ranges $\rightarrow \{0, \text{LD}, 1\}$), and evidence template.

9.2 Sign Registry Detail

Requirement 9.2 (Minimal Sign Set). **[REQ-9.2]** The epoch-0 Sign Registry MUST define at minimum the following role mappings:

Role	Codepoint(s)	Translit.	Babylon Equivalent
FN	U+12000 (`A`)	`a`	`function`
IF	U+12157 (`TUKUL`)	`tukul`	`if`
ELSE	U+12038 (`BAL`)	`bal`	`else`
RETURN	U+1202D (`AŠ`)	`aš`	`return`
LET	U+120FB (`NAM`)	`nam`	`let`
PRINT	U+1214E (`TA`)	`ta`	`print`
TRUE	U+12079 (`DU`)	`du`	`true`
FALSE	U+12080 (`DUG`)	`dug`	`false`

Rationale 9.1. The specific codepoint assignments are provisional for epoch-0. They serve as normative test fixtures. Future epochs may reassign signs through the mutation/crystallization process, provided the reassignment satisfies all gate criteria and produces a valid crystal.

9.3 Digest Binding

Requirement 9.3 (Registry Digest Binding). **[REQ-9.3]** All active registries MUST be bound by SHA-256 digest in the run descriptor and surfaced in emitted manifests. The

digest is computed by: serialize registry to canonical JSON (RFC 8785), then SHA-256 hash. This uses the existing `glyph_registry::compute_digest()` function.

9.4 Registry Evolution

Requirement 9.4 (Registry Immutability During Build). **[REQ-9.4]** Registries MAY evolve between language epochs. A build MUST NOT reinterpret itself under a moving registry target. Registry mutation belongs to evolution time (section 5.4), not to build time.

10 Language Epochs

10.1 Definition

Definition 10.1 (Language Epoch). A language epoch E_t is the frozen, reproducible semantic state of COL at a particular point in its lineage. It includes the active registry digests, policy set, lineage metadata, and crystallization status. An epoch is a hashable, versionable, replayable JSON object.

10.2 Epoch Object

Requirement 10.1 (Epoch Structure). **[REQ-10.1]** An epoch object MUST contain at minimum:

Listing 2: Epoch Object Schema

```
{
  "epoch_id": "col-epoch-NNNN",
  "parent_epoch": "col-epoch-MMMM | null",
  "sign_registry_digest": "<sha256>",
  "macro_registry_digest": "<sha256>",
  "profile_registry_digest": "<sha256>",
  "obligation_registry_digest": "<sha256>",
  "gate_registry_digest": "<sha256>",
  "policy_set": { ... },
  "lineage": { "parents": [...], "depth": N },
  "created_at": "<ISO 8601>",
  "epoch_digest": "<sha256 of canonical self>"
}
```

The `epoch_digest` is computed by serializing the epoch object with `epoch_digest=""` to canonical JSON and hashing with SHA-256, analogous to `RunDescriptor.compute_run_id()`.

10.3 Replay Stability Across Epochs

Requirement 10.2 (Replay Stability). **[REQ-10.2]** Replay stability requires that earlier epochs remain reconstructible and verifiable as earlier epochs. An artifact emitted under E_t MUST remain verifiable under E_t's registry digests, even if E_{t+1} exists. New epochs MUST NOT overwrite or destroy old epoch objects.

11 Mutation Engine

11.1 Candidate Generation

Requirement 11.1 (Deterministic Candidate Generation). **[REQ-11.1]** The mutation engine generates candidates from a given epoch E_t. A candidate MAY consist of: a new glyph-role binding, a modified macro expansion, a changed gate threshold, a new profile weighting, or another admissible registry transformation. Candidate generation MUST be deterministic under the governing seed, run descriptor, and policy context. An identical evolution run under identical conditions MUST produce the same candidates in the same order.

11.2 Evaluation Metrics

Requirement 11.2 (Structured Evaluation). **[REQ-11.2]** Candidate evaluation MUST use a structured metric space, not a single scalar score. The minimum metric set is:

- **Coherence** (κ): internal consistency of the candidate registry set.
- **Entropy** (H): information-theoretic measure of registry distribution.
- **Drift** (δ): semantic distance from the parent epoch.
- **Path-Invariance Gap** (π): discrepancy between alternative expansion paths.
- **Compression** (γ): semantic density gain from the mutation.
- **Resonance Fitness** (ρ_f): coherence relative to the active resonance model (section 16).
- **Stability** (σ): convergence behavior under repeated evaluation.

All metrics MUST be computed in Q16 fixed-point arithmetic to preserve determinism.

11.3 Mutation Formalization

A mutation is a directed transformation along a semantic gradient:

$$M_{t+1} = M_t + \Delta(\nabla\psi, \rho, \omega, RD)$$

where ψ denotes semantic weighting, ρ coherence/resonance density, and ω phase ordering. The direction depends on seed-, policy-, and registry-bound rules, not on uncontrolled randomness.

11.4 Mutation Manifest

Requirement 11.3 (Mutation Manifest). **[REQ-11.3]** Every evolution run MUST emit a mutation manifest documenting: source epoch, candidate set, active search and gate parameters, evaluation results per candidate, accept/reject/latent decision per candidate, and the newly generated epoch ID (if any). The manifest MUST be hashable and includable in the bundle.

12 Tripolar Gate Logic

12.1 Three Decision States

Requirement 12.1 (Tripolar States). **[REQ-12.1]** COL defines a tripolar decision layer with three states:

`0` Reject. The candidate is inadmissible, unstable, or incompatible with active obligations.

`LD` Latent Decision. The candidate is neither clearly accepted nor clearly rejected. It remains in a controlled intermediate state pending further evaluation or crystallization.

`1` Accept. The candidate satisfies all active gate and obligation criteria and is admitted to crystallization.

The latent state `LD` is indispensable because language mutation often produces structures that are promising yet not ready for adoption.

12.2 Gate Semantics

A gate does not merely approve or deny; it classifies the current semantic maturity of a candidate. This enables disciplined transition from mutation to crystallization without forcing premature acceptance or premature deletion.

12.3 Gate Registry Integration

Requirement 12.2 (Gate Registry). **[REQ-12.2]** Gate behavior MUST be registry-defined, not hardcoded. Gate definitions include: input metrics, threshold functions, tripolar mapping rules, and evidence templates. Gate outcomes MUST be surfaced in manifests or evidence objects. The Gate Registry MUST be digest-bound in the run descriptor.

12.4 Gate Object

A gate evaluation produces a gate object:

Listing 3: Gate Object Schema

```
{
  "candidate_id": "<hash>",
  "gate_id": "<registry ref>",
  "metrics": {
    "coherence": <Q16>,
    "drift": <Q16>,
    "compression": <Q16>,
    "resonance_fitness": <Q16>,
    "stability": <Q16>
  },
  "tripolar_state": "0 | LD | 1",
  "evidence_digest": "<sha256>"
}
```

13 Contraction, Fixpoints, and Condensation

13.1 Contractive Update Principle

Requirement 13.1 (Contraction). **[REQ-13.1]** COL assumes that meaningful stabilization requires local or regional contraction. If updates in a relevant region satisfy

$$d(T(x), T(y)) \;\leq\; c \cdot d(x, y), \qquad 0 \leq c < 1,$$

then repeated application of admissible updates tends toward a stable state. The language does not require global contraction of all imaginable mutations, only contraction in those spaces where stabilization is claimed.

13.2 Banach Fixpoint Interpretation

A candidate that converges to a unique fixed point under admissible update rules possesses a stronger claim to stability than a candidate selected only once. A fixpoint is not metaphysical truth but a technically sufficient anchor for crystallization.

Requirement 13.2 (Fixpoint Evidence)**.** **[REQ-13.2]** For any candidate presented for crystallization, the system MUST demonstrate that repeated evaluation under the active update rules converges within a bounded number of iterations. The convergence trace MUST be recorded and includable in the evidence chain.

13.3 Micro-Dynamics Convergence

Language epochs frequently emerge through many small local shifts: a registry binding changes, a macro is refined, a gate rule is tightened. COL MUST demonstrate not only global convergence but also local convergence of individual micro-dynamic steps.

13.4 Condensation

Definition 13.1 (Condensation)**.** Condensation is the transition from a distributed, still-moving candidate structure to a stabilized semantic form. A condensed state is characterized by: (a) its core meanings no longer oscillate, (b) its lowering is stable, (c) its gate states no longer flip between LD and 1, (d) its evidence structure is complete and replay-stable.

14 HDAG Ordering and Invariance

14.1 Ordered Language Lineage

Requirement 14.1 (Acyclic Lineage)**.** **[REQ-14.1]** Language evolution MUST be representable as a directed acyclic graph (DAG). An epoch MAY have one or more parent epochs, but cycles that undermine semantic ordering are forbidden. Without acyclicity, it would be impossible to determine which registry lineage governs an artifact.

14.2 Cross-Epoch Invariants

Requirement 14.2 (Invariant Preservation)**.** **[REQ-14.2]** The following properties MUST survive across all epoch transitions:

- Deterministic lowering (Invariant 1.1)
- Digest-bound registries (Invariant 1.4)
- Bundle and manifest compatibility (Invariant 1.8)
- Replay and verification of already-emitted artifacts (Invariant 1.5)

Additional invariants MAY be imposed per evolution regime, including preservation of specific macro operator identities, path equivalence stability, and bounded semantic drift per epoch.

14.3 Self-Compilation Stability

Requirement 14.3 (Self-Verification)**.** **[REQ-14.3]** A mature epoch SHOULD be able to verify its own lineage: its active registries, the relevant subset of earlier epochs, and the mutation/crystallization chain that produced it. An epoch that cannot process the conditions of its own evolution is structurally fragile.

15 Crystallization Layer

15.1 Mutation as Observation Stream

Candidate mutations are first treated as an observation stream:

$$O = (m_1, m_2, \ldots, m_n)$$

where each m_i is an evidence-capable mutation unit. The system evaluates repeated structures, regularities, and stabilization tendencies before admitting a new epoch.

15.2 Crystal Formation Criteria

Requirement 15.1 (Crystal Formation). **[REQ-15.1]** A candidate MAY be recognized as a crystal only if:

(i) it possesses a complete evidence chain from candidate generation through gate decision,
(ii) its registry digests are stable (no further drift under repeated evaluation),
(iii) its update behavior is convergent or contractively stabilized,
(iv) it passes bundle verification under replay.

15.3 Crystal Manifest

Requirement 15.2 (Crystal Manifest). **[REQ-15.2]** A crystal manifest MUST bind:

- Crystal ID (content hash of the crystal manifest itself)
- Parent epoch ID
- All five registry digests
- TIC-related stability information (convergence metrics)
- Gate decision evidence digest
- Mutation manifest reference

15.4 MEF and Evidence Chain

Every phase of epoch formation MUST be recorded as a tamper-evident step in the MEF chain: which candidate set was evaluated, which gates were applied, which states were 0/LD/1, which registry digests were adopted, and which manifest binds the resulting epoch. This uses Babylon's existing `glyph-mef::MefChain` with evolution-specific `OperatorEvidence` entries.

16 Resonance Layer

16.1 Operators as Resonance Network

The resonance layer treats glyph operators, macro clusters, and registry states as a structured network with phase-like coupling relations. This does not replace deterministic logic; it adds an analytical view for evaluating coherence beyond syntax alone.

16.2 Resonance Gating

Requirement 16.1 (Resonance as Metric). **[REQ-16.1]** Resonance fitness MAY be incorporated into gate decisions as one of the evaluation metrics (section 11.2). A candidate MAY be rejected, held latent, or accepted depending on how coherent it is relative to the active resonance model. Example mapping:

- Low resonance → contributes toward Reject (`0`)
- Medium resonance with high drift → contributes toward Latent (`LD`)
- High resonance with stable path → contributes toward Accept (`1`)

Resonance gating MUST NOT override hard invariants.

16.3 Spectral Compression

When multiple operator paths form a stable resonance cluster, they MAY be compressed into a new macro operator, provided path-invariance and replay conditions are preserved. This constitutes an additional source of semantic densification beyond explicit rule design.

17 Spiral Embedding Model (Optional)

17.1 Spiral Coordinates

For implementations that support it, operators, registry candidates, and language states MAY be embedded into a spiral coordinate space with at least three axes:

- ψ: semantic weighting,
- ρ: coherence or resonance density,
- ω: phase or rhythmic ordering.

These coordinates serve as evaluation axes for mutation candidates. They are computed in Q16 fixed-point and stored as part of the mutation evaluation record.

17.2 Convergence Conditions

A mutation is admissible only if it satisfies explicit convergence bounds: bounded drift, coherent registry relations, sufficient resonance stability, reproducible lowering, and gate compatibility. These bounds MAY be expressed as constraints on the spiral coordinates.
Rationale 17.1. The spiral model was introduced in v2.1 as an alternative to flat metric vectors. v3.0 preserves it as an optional analytical frame. Implementations that prefer flat metric vectors MAY compute the same metrics without spiral embedding, provided all evaluation metrics from section 11.2 are available.

18 QLOGIC-ISA Layer (Optional Extension)

18.1 Spectral Opcodes

COL MAY optionally access a second semantics layer in which macros or operator clusters reference spectral opcodes. Candidate opcode classes include: `OSC`, `HARM`, `MIX`, `GATE`, `KICK_U`, `KICK_V`, `CONDENSE`, `MEASURE`, `DECIDE`, `LEARN`.

18.2 Multi-Stage Addressing

The multi-stage chain is:

$$\text{Glyph} \rightarrow \text{Registry Role} \rightarrow \text{Macro Subgraph} \rightarrow \text{ISA Opcode (optional)}$$

The surface remains compact while internal semantics can be arbitrarily deep.

18.3 Path Invariance

Requirement 18.1 (ISA Path Invariance)**.** **[REQ-18.1]** If different glyph sequences or macro expansions converge on the same condensed semantic state, this MUST hold only under explicit path invariance. Ambiguity is permitted only where it resolves to the same verifiable form.

Rationale 18.1. The QLOGIC-ISA layer is not required for v3.0 Definition of Done. It is preserved as a designed extension point for future epochs that need runtime-close operator semantics beyond AST-level IR nodes.

19 Evidence Capsules

19.1 Definition

Definition 19.1 (Evidence Capsule)**.** An evidence capsule is a declared, bounded, manifest-bound data object transported through the compilation chain. It is not a hidden channel and not an unstructured comment. It is part of the verifiable machine state.

19.2 Capsule Policy

Requirement 19.1 (Capsule Policy)**.** **[REQ-19.1]** Every capsule MUST be governed by an explicit policy defining:

- Maximum size (in bytes)
- Admissible content classes (context-anchor, commitment, boundary-declaration, attachment)
- Provenance requirements
- Hash binding method
- Emission constraints

Capsules are only admissible if their existence and content do not destroy replay stability.

19.3 Capsule Object

Listing 4: Capsule Object Schema

```
{
  "capsule_id": "<sha256>",
  "capsule_type": "context-anchor | commitment | boundary | attachment",
  "size_limit": 2048,
  "content_digest": "<sha256>",
  "manifest_digest": "<sha256>",
  "policy_ref": "<registry ref>",
  "provenance": "<origin chain>"
}
```

19.4 Verify and Replay

Requirement 19.2 (Capsule Verification)**.** **[REQ-19.2]** Capsules MUST be fully manifested in the bundle. Verification MUST check: (a) capsule presence, (b) hash match with manifest, (c) policy binding validity, (d) replay stability of the build with the capsule.

20 Canonical IR and Lowering Contract

20.1 Single Canonical Target

Requirement 20.1 (IR Target). **[REQ-20.1]** Every valid COL source MUST lower into a single `glyph_ir::IrDocument`. The IR uses the existing schema version `1.0.0` with the following node kinds: `Module`, `Function`, `Block`, `If`, `Return`, `Call`, `Assignment`, `Literal`, `BinaryOp`, `Identifier`. Edge kinds: `Contains`, `Next`, `Condition`, `ThenBranch`, `ElseBranch`, `Argument`, `Target`, `Value`, `Left`, `Right`, `CalleeRef`.

20.2 Extended Metadata

Requirement 20.2 (Evolution Metadata). **[REQ-20.2]** COL-emitted IR documents or their bundle metadata MUST include:

- `epoch_id`
- `sign_registry_digest`, `macro_registry_digest`, `profile_registry_digest`, `obligation_registry_gate_registry_digest`
- `mutation_lineage` (reference to parent epoch chain)
- `crystal_id` (if crystallized)
- `capsule_ids` (list of active capsule references)

These fields MAY be stored in the `IrNode.properties` map of the root Module node or as separate manifest entries. They MUST be deterministically produced and hashable.

20.3 Stable Lowering

Requirement 20.3 (Stable Lowering Chain). **[REQ-20.3]** Stable lowering means:

(i) The same glyph sequence under the same registry configuration produces the same token sequence.
(ii) The same token sequence produces the same parse tree.
(iii) The same parse tree produces the same IR nodes, edges, and ordering.
(iv) The same metadata produces the same manifest digest.

Evolvability MUST NOT cause a previously fixed build to receive a different lowering retroactively.

20.4 Deterministic ID Generation

Requirement 20.4 (Content-Hash IDs). **[REQ-20.4]** Node identifiers MUST be generated using the content-hash scheme: `content_hash_node_id(kind, name, scope_path)` → SHA-256 digest. This is the existing scheme in Babylon's `glyph_canon::content_hash_node_id()`.

20.5 Backward Compatibility

Requirement 20.5 (Epoch-Scoped Compatibility). **[REQ-20.5]** A new epoch MAY introduce new constructs, macros, gates, and registry structures without destroying the verifiability of earlier artifacts. Backward compatibility means:

- Old artifacts remain verifiable under their original epoch.
- Old epochs remain available as objects.
- Old digests and bundles remain reconstructible.
- New epochs MAY reference but MUST NOT overwrite earlier epochs.

21 Integration into Babylon Compiler

21.1 Frontend Architecture

Requirement 21.1 (Frontend Modules). **[REQ-21.1]** The cuneiform frontend MUST provide the following modules under `glyph-frontends::cuneiform`:

`lexer` Unicode-aware tokenization of cuneiform signs and structural tokens. Token type: `SIGN(cp: u32)`.
`parser` AST construction per the EBNF (section 8.2).
`lower` Deterministic transformation of AST to `IrDocument` using `content_hash_node_id`.
`registry_adapter` Resolution of sign codepoints to roles and macro patterns against the active Sign and Macro registries.

21.2 CLI Extensions

Requirement 21.2 (CLI Commands). **[REQ-21.2]** The `glyph-cli` MUST be extended with:

Listing 5: COL CLI Surface

```
# Standard build (build time)
glyph run --frontend cuneiform --epoch <epoch-id> --rd run.json -i input.col -o ./
    bundle

# Verification
glyph verify --bundle ./bundle

# Dump IR
glyph dump-ir --frontend cuneiform -i input.col

# Evolution (evolution time)
glyph evolve-registry --epoch <epoch-id> --rd run.json --seed <seed>

# Crystallization (crystallization time)
glyph crystallize-epoch --candidate <candidate-id> --epoch <parent-epoch>

# Inspection
glyph inspect-epoch --epoch <epoch-id>
```

21.3 Bundle Format v3.0

Requirement 21.3 (Extended Bundle). **[REQ-21.3]** In addition to the standard bundle artifacts (`source.norm.txt`, `ir.json`, `trace.json`, `manifest.json`, `mef.json`, `evidence.json`), the COL bundle MUST carry:

- `epoch-manifest.json` — the active epoch object
- `mutation-manifest.json` — if the build is an evolution run
- `registry-lineage.json` — directed lineage from epoch-0 to current
- `crystal-manifest.json` — if a crystal was produced
- `proof-objects/` — directory of gate and fixpoint evidence
- `capsules/` — directory of evidence capsule objects

All additional files MUST be included in the manifest's `artifact_digests` map.

22 Test Architecture

Requirement 22.1 (Six Test Families)**.** **[REQ-22.1]** The following test families are normative:

22.1 T1: Determinism Tests

Identical source + identical run descriptor + identical epoch MUST produce identical: IR digest, manifest digest, trace digest, crystal references, and gate decision objects. This is the foundational test: if it fails, nothing else matters.

22.2 T2: Evolution Tests

The system MUST demonstrate: (a) generation of a candidate from a base epoch, (b) evaluation of that candidate, (c) formation of a new epoch, (d) proof that the original epoch remains replay-stable.

22.3 T3: Tripolar Gate Tests

For defined candidate sets, the system MUST deterministically assign states `0`, `LD`, or `1`. Boundary cases and repeated evaluation runs are required. The latent state `LD` MUST NOT become a residual category; it MUST be a controlled intermediate state reproducible across runs.

22.4 T4: Contraction and Fixpoint Tests

For defined update families, the system MUST show: (a) repeated updates converge, (b) resulting states become unique, (c) drift remains within defined bounds.

22.5 T5: Crystal Tests

When an epoch is declared a crystal: (a) the evidence chain MUST be complete, (b) registry digests MUST be correctly bound, (c) the crystal ID MUST be stably reproducible, (d) `glyph verify` MUST reconstruct the crystal object.

22.6 T6: Capsule Tests

If capsules are enabled: (a) size and policy bounds MUST be enforced, (b) hash binding MUST be correct, (c) replay stability MUST hold with capsules present, (d) violation cases (oversized, wrong type, missing provenance) MUST be rejected.

22.7 Normative Test Vectors

Requirement 22.2 (Minimum Test Vectors)**.** **[REQ-22.2]** At least three test programs MUST serve as normative reference cases:

TV1 Minimal print. A single function calling `print` with a string literal. Must produce: Module, Function, Block, Call, Literal nodes. Verify: deterministic IR digest.

TV2 Conditional. A function with if/else and a comparison. Must produce: If, BinaryOp, ThenBranch, ElseBranch edges. Verify: deterministic IR digest.

TV3 Macro expansion. A glyph or glyph cluster that expands to a non-trivial IR subgraph. Must demonstrate $D > 1.0$. Verify: expansion record in trace, deterministic post-expansion digest.

23 Definition of Done

Requirement 23.1 (Acceptance Criteria). **[REQ-23.1]** Implementation of COL v3.0 is considered complete only if *all* of the following hold:

DoD-1 Frontend complete. COL sources compile through Babylon under a `cuneiform` frontend. Glyphs are deterministically tokenized. Parsing and lowering produce a valid `IrDocument`. The command `glyph run --frontend cuneiform` emits IR and a bundle. The command `glyph verify` passes.

DoD-2 Deterministic lowering. Identical inputs produce identical IR bytes, manifest digests, and trace digests across repeated runs. T1 tests pass.

DoD-3 Registry binding. All five registries (Sign, Macro, Profile, Obligation, Gate) are digest-bound in the run descriptor and surfaced in the manifest. Changing a registry entry changes the digest.

DoD-4 Epoch system operational. At least two language epochs exist as digest-bound objects. Each epoch possesses its own registry set. Digests are reproducible. Earlier epochs are not destroyed by later ones.

DoD-5 Mutation engine operational. At least one non-trivial registry mutation has been deterministically generated, evaluated, and transformed into a new epoch. Purely manual registry edits do not satisfy this criterion.

DoD-6 Tripolar gating operational. Mutations and candidates are deterministically classified into `0`, `LD`, or `1`. The state `LD` has a functional role in the crystallization chain. T3 tests pass.

DoD-7 Crystallization demonstrated. At least one mutation has been accepted and crystallized into a stable language epoch. The crystal manifest is complete: candidate provenance, gate decision, registry digests, verify capability. T5 tests pass.

DoD-8 Non-triviality proved. At least one glyph or glyph cluster expands into a non-trivial, evidence-bearing operator subgraph with $D > 1.0$. This expansion is visible in the bundle, manifest, and verify process. TV3 passes.

DoD-9 Bundle complete. Bundles emit all required artifacts: normalized source, IR, trace, manifest, MEF chain, evidence, epoch manifest, registry lineage. Optional: mutation manifest, crystal manifest, capsules.

DoD-10 Replay stable. An artifact produced under epoch E_0 remains verifiable after E_1 exists. T2 tests pass.

DoD-11 Contraction demonstrated. At least one stabilizing candidate path contracts toward a reproducible fixpoint. T4 tests pass.

DoD-12 All test families pass. T1–T6 and test vectors TV1–TV3 pass without exception.

24 Obsolescence of Predecessor Documents

The three predecessor documents are rendered historically obsolete by this specification for the following reasons:

v1.0 (COL_de, German outline)
All content is absorbed. Placeholders in appendices A–F are replaced by concrete schemas, EBNF, registry definitions, test vectors, and JSON examples. The static model (no evolution) is superseded by the full epoch/mutation/crystallization framework. The German-language formulation is unified with the English normative text.

v2.0 (English specification)
All invariants, design principles, registry taxonomy, epoch model, mutation engine, tripolar gate logic, contraction/fixpoint analysis, crystallization layer, resonance layer, evidence capsules, IR contract, integration plan, test architecture, and Definition of Done are absorbed verbatim or strengthened. The discursive discussion and conclusion sections are absorbed into rationale blocks. Example objects (epoch, gate, capsule) are refined and schema-specified.

v2.1 (Keilschrift++)
All extensions—spiral embedding, QLOGIC-ISA, detailed Banach-fixpoint interpretation, micro-dynamics convergence, HDAG ordering, self-compilation stability, MEF/evidence chain for crystallization, RSS resonance with phase/twist, tightened DoD—are absorbed. The spiral model and QLOGIC-ISA are preserved as optional extension points rather than mandatory requirements, resolving the over-specification risk in v2.1. The detailed German-language rationale for cultural/historical positioning of cuneiform is absorbed into the invariant framework (cuneiform is technically relevant only as an operator address space, not as cultural artifact).

No normative content from any predecessor document is lost. Every invariant is preserved or strengthened. Every concrete artifact (EBNF, epoch schema, gate schema, capsule schema, test vectors) is present in this document. The three predecessors serve only as historical evolution records of the specification process.

A Epoch Object Example

```
{
  "epoch_id": "col-epoch-0001",
  "parent_epoch": null,
  "sign_registry_digest": "a3f8...c4d1",
  "macro_registry_digest": "b7e2...91f0",
  "profile_registry_digest": "c1d4...e3a2",
  "obligation_registry_digest": "d9f0...b1c3",
  "gate_registry_digest": "e2a1...d4f5",
  "policy_set": {
    "unicode_normalization": "NFC",
    "strict_mode": true,
    "seed_mode": "fixed",
    "seed": "0"
  },
  "lineage": {
    "parents": [],
    "depth": 0
```

```
  },
  "created_at": "2026-03-17T00:00:00Z",
  "epoch_digest": "f1b2...a3c4"
}
```


B Mutation Manifest Example


```
{
  "source_epoch": "col-epoch-0001",
  "candidates": [
    {
      "candidate_id": "cand-001",
      "mutation_type": "new_glyph_binding",
      "description": "Bind U+12100 to LOOP operator",
      "metrics": {
        "coherence": 60293,
        "entropy": 45102,
        "drift": 4587,
        "path_invariance_gap": 0,
        "compression": 53674,
        "resonance_fitness": 58201,
        "stability": 62441
      },
      "tripolar_state": "1"
    }
  ],
  "gate_parameters": {
    "gate_registry_digest": "e2a1...d4f5",
    "active_gates": ["gate-coherence-v1", "gate-drift-v1"]
  },
  "result_epoch": "col-epoch-0002",
  "manifest_digest": "a1b2...c3d4"
}
```


C Gate Object Example


```
{
  "candidate_id": "cand-001",
  "gate_id": "gate-coherence-v1",
  "metrics": {
    "coherence": 60293,
    "drift": 4587,
    "compression": 53674,
    "resonance_fitness": 58201,
    "stability": 62441
  },
  "tripolar_state": "1",
  "evidence_digest": "b2c3...d4e5"
}
```


D Crystal Manifest Example

```
{
  "crystal_id": "crystal-0001",
  "epoch_id": "col-epoch-0002",
  "parent_epoch": "col-epoch-0001",
  "sign_registry_digest": "a3f8...c4d1",
  "macro_registry_digest": "c8e2...91f0",
  "profile_registry_digest": "c1d4...e3a2",
  "obligation_registry_digest": "d9f0...b1c3",
  "gate_registry_digest": "e2a1...d4f5",
  "convergence_iterations": 7,
  "contraction_factor": 39321,
  "evidence_chain_digest": "d3e4...f5a6",
  "mutation_manifest_digest": "a1b2...c3d4",
  "crystal_digest": "e4f5...a6b7"
}
```


E Capsule Object Example


```
{
  "capsule_id": "cap-001",
  "capsule_type": "commitment",
  "size_limit": 2048,
  "content_digest": "f5a6...b7c8",
  "manifest_digest": "a6b7...c8d9",
  "policy_ref": "capsule-policy-v1",
  "provenance": "epoch:col-epoch-0001/gate:gate-coherence-v1"
}
```


F Sign Registry v1 Example


```
{
  "schema_version": "1.0.0",
  "type": "sign_registry",
  "epoch_id": "col-epoch-0001",
  "entries": [
    {
      "sign": "U+12000",
      "cluster": null,
      "role": "FN",
      "operator_id": "op-fn-v1",
      "transliteration": "a",
      "version": "1.0.0"
    },
    {
      "sign": "U+12157",
      "cluster": null,
      "role": "IF",
      "operator_id": "op-if-v1",
      "transliteration": "tukul",
      "version": "1.0.0"
    },
    {
```

```
      "sign": "U+12038",
      "cluster": null,
      "role": "ELSE",
      "operator_id": "op-else-v1",
      "transliteration": "bal",
      "version": "1.0.0"
    },
    {
      "sign": "U+1202D",
      "cluster": null,
      "role": "RETURN",
      "operator_id": "op-return-v1",
      "transliteration": "as",
      "version": "1.0.0"
    },
    {
      "sign": "U+120FB",
      "cluster": null,
      "role": "LET",
      "operator_id": "op-let-v1",
      "transliteration": "nam",
      "version": "1.0.0"
    },
    {
      "sign": "U+1214E",
      "cluster": null,
      "role": "PRINT",
      "operator_id": "op-print-v1",
      "transliteration": "ta",
      "version": "1.0.0"
    },
    {
      "sign": "U+12079",
      "cluster": null,
      "role": "TRUE",
      "operator_id": "op-true-v1",
      "transliteration": "du",
      "version": "1.0.0"
    },
    {
      "sign": "U+12080",
      "cluster": null,
      "role": "FALSE",
      "operator_id": "op-false-v1",
      "transliteration": "dug",
      "version": "1.0.0"
    }
  ]
}
```


G Normative Test Vector TV1: Minimal Print

Source (cuneiform with transliteration):

```
<U+12000> <U+1214E>_main() {
```

```
    <U+1214E>("namaste");
}
```

Transliterated equivalent:

```
a ta_main() {
    ta("namaste");
}
```

Expected IR nodes: Module(root), Function(ta_main), Block, Call(print), Literal("namaste"), Identifier(print).

Expected edges: Module $\xrightarrow{\text{contains}}$ Function, Function $\xrightarrow{\text{contains}}$ Block, Block $\xrightarrow{\text{contains}}$ Call, Call $\xrightarrow{\text{callee_ref}}$ Identifier, Call $\xrightarrow{\text{argument}}$ Literal.

Determinism check: Two runs with identical RD and epoch-0 must produce identical IR digests.

H Run Descriptor Extension for COL

The `RunDescriptor` struct gains the following fields for COL:

```
{
  "...existing fields...",
  "epoch_id": "col-epoch-0001",
  "registries": {
    "operator": "<sha256>",
    "macro": "<sha256>",
    "obligation": "<sha256>",
    "observable": "<sha256>",
    "sign": "<sha256>",
    "profile": "<sha256>",
    "gate": "<sha256>"
  }
}
```

The existing four registry digest fields are preserved for backward compatibility with Sanskroot and HanLan. The three new fields (`sign`, `profile`, `gate`) are added for COL and default to all-zeros when unused.

I Glossary

Build Time Ordinary compilation under a fixed epoch and fixed registries.

Capsule Declared, bounded boundary object carried through the compilation chain.

Condensation Transition from a moving candidate structure to a stabilized form.

Crystal Stabilized, evidence-bound epoch form with complete provenance.

Epoch Frozen, reproducible language version bound to a specific registry set.

Evolution Time Generation and evaluation of candidate mutations.

Crystallization Time
Stabilization of an accepted candidate into a new epoch.

HDAG Directed acyclic graph of language epoch lineage.

LD Latent Decision — tripolar intermediate state between rejection and acceptance.

MEF Morphogenetic Evidence Framework — tamper-evident hash chain.

Q16 Fixed-point arithmetic: signed 32-bit, 1/65536 resolution.

Registry Lineage
Directed provenance structure of language and registry versions.

Semantic Density
Ratio of IR nodes to surface tokens after macro expansion.

Sign A cuneiform Unicode character treated as an atomic semantic token.

TIC Temporal Information Certificate — convergence tracking object.

Tripolar Gate Decision layer classifying candidates as 0 (reject), LD (latent), or 1 (accept).